## 26. Design a Data Structure with add(x) and removeAndPrint(x)

### Problem Statement

Design a data structure supporting two operations:

- add(int x):
  - Adds integer `x` to the data structure.
- removeAndPrint(int x):
  - Removes **all occurrences** of `x`
  - Prints all remaining elements
  - Maintains **insertion order**

---

## Example

Operations:

add(10)
add(20)
add(10)
add(30)

removeAndPrint(10)
Before removal: 10 20 10 30
Remove all `10`s: 20 30
Print: 20 30
Data structure becomes: 20 30

---

Continue:
add(30)
add(40)
add(60)

removeAndPrint(30)
Before removal: 20 30 30 40 60
Remove all `30`s: 20 40 60
Print: 20 40 60

---

## Requirements

| Operation | Requirement |
|---|---|
| add(x) | Efficient insertion |
| removeAndPrint(x) | Remove all occurrences of x |
| Printing | Maintain insertion order |

---

# Solution 1: ArrayList

### add(x)
list.add(x);
Complexity: Amortized O(1)

---

## removeAndPrint(x)
**Naive approach:**

```
for(each element){
  if(element == x)
     remove(element);
}
```

Problem: Removing shifts elements.

Worst case: 10 10 10 10 10

***Removing:***

1st removal → shift n−1 elements
2nd removal → shift n−2 elements
...
Complexity: $O(n^2)$

---

## Better ArrayList Solution
Use an auxiliary array.

### Algorithm
Traverse original list.

If element ≠ x:
Print it
Store it in auxiliary list

After traversal: Replace original list with auxiliary list.

Complexity: O(n)
Extra Space: O(n)

---

### Drawbacks of ArrayList

| Issue | Reason |
|---|---|
| add() | Amortized O(1) |
| removeAndPrint() | $O(n^2)$ naive |
| O(n) solution | Needs extra space |
| Implementation | Slightly tricky |

---

# Solution 2: LinkedList
## Why LinkedList?

Because: Iterator.remove() → O(1)
after reaching the node.

---

## Data Structure
LinkedList<Integer> list = new LinkedList<>();

---

## add(x)
list.addLast(x);
Complexity: O(1) Worst Case.

---

## removeAndPrint(x)
Use Iterator.

---

## Algorithm

Traverse list.

For every element:

**If element == x**

it.remove();

Remove it.

---

**Otherwise**
System.out.print(element);
Print it.

---

### Why Iterator?
Because Iterator allows:

| Operation | Supported |
|---|---|
| Traverse | Yes |
| Remove current element | Yes |
| Continue traversal | Yes |

---

## Iterator Working
Suppose: 10 20 10 30

Iterator starts:
↓
10 20 10 30

---

### First next()
Returns: 10
Since: 10 == x
Remove: it.remove();
List becomes: 20 10 30

---

### Next iteration
Returns: 20
Since: 20 != x
Print: 20

---

### Next iteration
Returns: 10
Remove: it.remove();
List becomes: 20 30

---

### Final iteration
Returns: 30
Print: 30

---

Output: 20 30

---

### Implementation
```
class DataStructure {

  LinkedList<Integer> list = new LinkedList<>();

  void add(int x){
     list.addLast(x);
  }

  void removeAndPrint(int x){

     Iterator<Integer> it = list.iterator();

     while(it.hasNext()){
        int curr = it.next();

        if(curr == x){
           it.remove();
        }
        else{
           System.out.print(
                 curr + " "
           );
        }
     }
     System.out.println();
  }
}
```

---

## Time Complexity

### add(x)

list.addLast(x);
Complexity: O(1) Worst Case.

---

## removeAndPrint(x)

Traverse: n elements
Removal: O(1)
Print:O(1)
Total: O(n)

---

## Space Complexity

Additional Space: O(1)

---

## Comparison

| Data Structure | add(x) | removeAndPrint(x) | Extra Space |
|---|---|---|---|
| ArrayList (naive) | O(1)* | $O(n^2)$ | O(1) |
| ArrayList (auxiliary) | O(1)* | O(n) | O(n) |
| LinkedList + Iterator | O(1) | O(n) | O(1) |

* Amortized O(1)

---

## Why LinkedList Wins?

**Because:** ArrayList removal -> Requires shifting -> Expensive
**While:** LinkedList Iterator.remove() -> Adjust pointers -> O(1)

---

## Interview Takeaway

**When you need:** Frequent insertions + Frequent removals during traversal + Maintain insertion order

Think: LinkedList + Iterator

because it provides:

add() → O(1)
removeAndPrint() → O(n)
extra space → O(1)

which is the optimal solution for this problem using standard Java collections.